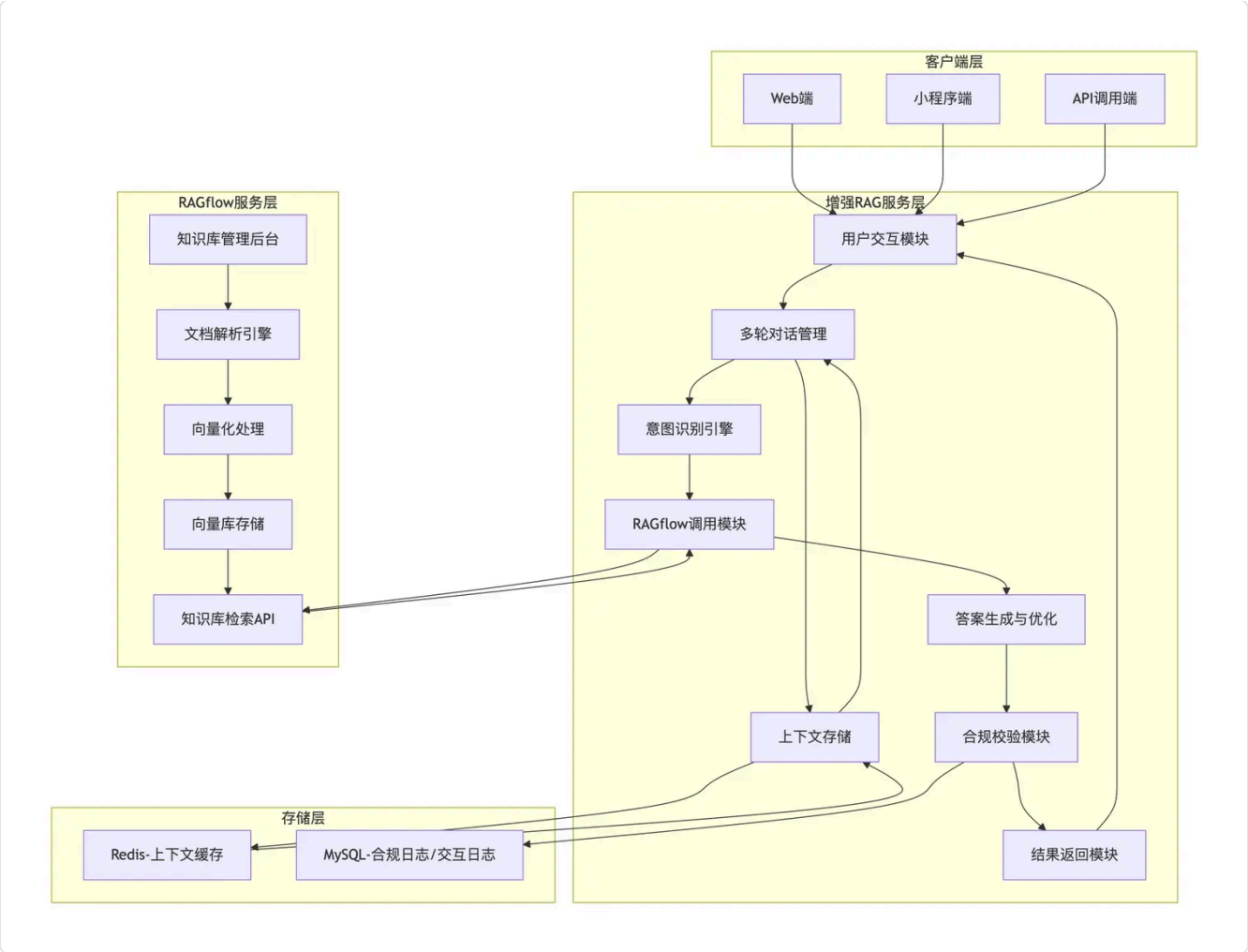
智保灵犀增强 RAG 调用系统产品技术方案

一、方案核心定位与目标

本方案聚焦“RAGflow 独立构建知识库 + 增强 RAG 服务负责交互与推理”的架构模式，实现“知识库管理与业务交互解耦”，核心目标：

1. **解耦部署**：RAGflow 专注知识库构建（文档解析、向量化、存储），增强 RAG 服务专注用户交互（多轮对话、意图识别、答案生成），各自独立迭代；
2. **交互增强**：支持保险场景多轮对话（如“甲状腺结节核保→保费多少→理赔条件”），答案精准溯源至 RAGflow 知识库，合规性 100%；
3. **轻量化适配**：兼容中小服务器部署（如 2 核 8G/4 核 16G），接口调用延迟≤1.5s，多轮对话上下文无丢失；
4. **保险场景适配**：内置重疾险 / 医疗险等领域 prompt 模板，支持合规校验、术语标准化，避免违规表述输出。

二、整体架构设计（解耦式双服务）


客户端层
Web端
小程序端
API调用端
RAGflow服务层
知识库管理后台
文档解析引擎
向量化处理
向量库存储
知识库检索API
增强RAG服务层
用户交互模块
多轮对话管理
意图识别引擎
RAGflow调用模块
答案生成与优化
上下文存储
合规校验模块
结果返回模块
存储层
Redis-上下文缓存
MySQL-合规日志/交互日志

系统架构图

Plain Text

```
flowchart TD
    subgraph 客户端层
        A[Web端]
        B[小程序端]
        C[API调用端]
    end

    subgraph 增强RAG服务层
        D[用户交互模块]
        E[多轮对话管理]
        F[意图识别引擎]
        G[RAGflow调用模块]
        H[答案生成与优化]
        I[合规校验模块]
        J[结果返回模块]
        K[上下文存储]
    end

    subgraph RAGflow服务层
        L[文档解析引擎]
        M[向量化处理]
        N[向量库存储]
        O[知识库检索API]
        P[知识库管理后台]
    end

    subgraph 存储层
        Q[Redis-上下文缓存]
        R[MySQL-合规日志/交互日志]
    end

    A --> D
    B --> D
    C --> D

    D --> E
    E --> F
    F --> G
```

```
    G --> O
    O --> G
    G --> H
    H --> I
    I --> J
    J --> D

    E --> K
    K --> E
    K --> Q
    Q --> K

    I --> R

    P --> L
    L --> M
    M --> N
    N --> O
```

核心交互流程

1. 用户通过 Web/ 小程序发起提问（如“重疾险甲状腺结节核保规则”）；
2. 增强 RAG 服务接收请求，解析用户意图 + 加载多轮对话上下文；
3. 调用 RAGflow 的检索 API，传入提问文本 + 保险领域过滤条件；
4. RAGflow 返回知识库匹配结果（文本片段 + 来源 + 相似度）；
5. 增强 RAG 服务结合上下文 + 领域 prompt 模板，优化答案生成；
6. 合规校验模块检测答案是否含违规表述（如“保证赔付”），通过后返回给用户；
7. 记录交互日志与合规日志，缓存多轮对话上下文（有效期 30 分钟）。

三、各模块技术设计

（一）RAGflow 部署与知识库配置（独立服务）

1. RAGflow 部署方案

RAGflow 是字节跳动开源的 RAG 知识库框架，支持文档解析、向量化、检索全流程，独立部署在服务器（可与增强 RAG 服务共用 2 核 8G 服务器，或单独部署）：

```bash
# 部署前提：Docker + Docker Compose（阿里云服务器预装）
# 1. 克隆RAGflow仓库
git clone https://github.com/infiniflow/ragflow.git
cd ragflow

# 2. 一键启动（轻量化配置，适配2核8G）
# 修改docker-compose.yml，调整资源限制：
# services:
#   ragflow-web:
#     deploy:
#       resources:
#         limits:
#           cpus: '1'
#           memory: 2G
#   ragflow-api:
#     deploy:
#       resources:
#         limits:
#           cpus: '1'
#           memory: 4G

# 3. 启动服务
docker-compose up -d

# 4. 访问RAGflow后台：http://服务器IP:9000（默认账号密码：admin@ragflow.com/123456）
```

2. 保险知识库配置（RAGflow 内操作）

配置项	具体操作	保险场景适配
文档上传	上传重疾险条款、核保规则、理赔指南等 PDF/Word	开启“智能分块”，按保险文 （责任 / 免责 / 等待期）分块
向量化模型	选择“bge-large-zh-v1.5”	适配中文保险术语，384/768 可选（优先 384 维省资源）
检索配置	开启“混合检索（关键词 + 语义）”	关键词检索匹配保险术语（如 期”），语义检索匹配模糊提问
知识库权限	设为“公开（仅本服务调用）”	仅允许增强 RAG 服务通过 AP 禁止外部访问
API 密钥生成	在“设置 -API 管理”生成调用密钥	用于增强 RAG 服务调用鉴权， 换

3. RAGflow 核心 API（增强 RAG 服务调用）

API 接口	请求方式	参数示例	返回示例
/api/v1/knowledge/search	POST	{"query":"甲状腺结节核保","knowledge_base_id":"xxx","top_k":3}	{"code":200,"data":{"documents":["匹配文本 配文本 2"],"metadatas":[{"source":"重疾险条款 .p
/api/v1/knowledge/list	GET	{"api_key":"xxx"}	{"code":200,"data":[{"id":"xxx","name":"重疾 库"}]}

（二）增强 RAG 服务设计（核心交互层）

1. 技术选型（轻量化 + 高性能）

模块	技术选型	选型理由
基础框架	FastAPI（Python 3.9）	异步性能优，接口文档自动生成，适 署
多轮对话管理	Redis 7.0	缓存上下文（Key：用户 ID，Value： 史），过期时间 30 分钟
意图识别	自定义规则 + 轻量 LLM（如 Qwen-1.8B）	保险场景意图有限（检索 / 咨询 / 理赔 + 小模型兼顾准确率与性能
答案生成	Prompt 模板 + 轻量 LLM	基于 RAGflow 检索结果，结合保险领 优化答案
合规校验	自定义规则引擎	内置保险违规词库（如“保证赔付” 理赔”），实时检测
部署方式	Docker 容器化	与 RAGflow 隔离，便于版本管理

2. 核心模块实现细节

（1）多轮对话上下文管理

Python

```python
import redis
import json
from datetime import timedelta

# 初始化Redis（适配2核8G，设置最大内存2G）
redis_client = redis.Redis(host='localhost', port=6379, db=0, maxmemory='2gb', maxmemory_policy='allkeys-lru')

# 存储对话上下文
def save_conversation_context(user_id: str, query: str, answer: str):
    key = f"context:{user_id}"
    # 获取历史上下文（无则初始化）
    history = redis_client.get(key)
    if history:
        history = json.loads(history)
    else:
        history = []
    # 追加新对话（最多保留10轮，避免内存溢出）
    history.append({"query": query, "answer": answer})
    if len(history) > 10:
        history = history[-10:]
    # 缓存30分钟
    redis_client.setex(key, timedelta(minutes=30), json.dumps(history))

# 获取对话上下文
def get_conversation_context(user_id: str):
    key = f"context:{user_id}"
    history = redis_client.get(key)
    return json.loads(history) if history else []
```

（2）RAGflow 调用模块（核心）

Python

```python
import requests
from pydantic import BaseModel

# RAGflow配置
RAGFLOW_API_URL = "http://服务器IP:9000/api/v1/knowledge/search"
RAGFLOW_API_KEY = "生成的API密钥"
KNOWLEDGE_BASE_ID = "重疾险知识库ID"

# 请求参数模型
class RAGflowQuery(BaseModel):
    query: str
    knowledge_base_id: str = KNOWLEDGE_BASE_ID
    top_k: int = 3
    api_key: str = RAGFLOW_API_KEY

# 调用RAGflow检索知识库
def call_ragflow(query: str) -> dict:
    try:
        req_data = RAGflowQuery(query=query).dict()
        response = requests.post(RAGFLOW_API_URL, json=req_data, timeout=5)
        if response.status_code == 200:
            return response.json()["data"]
        else:
            return {"error": f"RAGflow调用失败: {response.status_code}"}
    except Exception as e:
        return {"error": f"RAGflow调用异常: {str(e)}"}
```

（3）答案生成与合规校验

Python

```python
# 保险领域Prompt模板
INSURANCE_PROMPT_TEMPLATE = """
基于以下知识库内容，结合用户历史对话上下文，回答用户问题，要求：
1. 语言简洁，符合保险行业规范，避免专业术语过度堆砌；
2. 必须引用知识库内容，禁止编造信息；
3. 结尾添加合规声明：本内容仅供参考，不构成投保建议，具体以保险合同条款为准。

知识库内容：{knowledge_content}
历史上下文：{context}
用户问题：{query}
"""

# 合规违规词库
VIOLATION_WORDS = ["保证赔付", "100%理赔", "无风险", "稳赚", "必赔"]

# 答案生成（结合RAGflow结果+上下文）
def generate_answer(query: str, ragflow_result: dict, context: list):
    # 整理知识库内容
    knowledge_content = ""
    if "documents" in ragflow_result and ragflow_result["documents"]:
        for doc, meta in zip(ragflow_result["documents"], ragflow_result.get("metadatas", [])):
            knowledge_content += f"【来源：{meta.get('source', '未知')}】{doc}\n"
    else:
        knowledge_content = "暂无相关知识库内容"

    # 拼接Prompt
    prompt = INSURANCE_PROMPT_TEMPLATE.format(
        knowledge_content=knowledge_content,
        context=json.dumps(context, ensure_ascii=False),
        query=query
    )

    # 调用轻量LLM生成答案（示例：本地部署的Qwen-1.8B）
    # 注：无GPU可改用API调用（如通义千问/豆包）
    answer = call_light_llm(prompt)

```

```python
    # 合规校验
    for word in VIOLATION_WORDS:
        if word in answer:
            answer = answer.replace(word, "[违规表述已屏蔽]")
            # 记录违规日志
            save_compliance_log(query, answer, "违规表述屏蔽")

    return answer

# 轻量LLM调用（示例）
def call_light_llm(prompt: str) -> str:
    # 本地部署Qwen-1.8B示例（适配2核8G，CPU推理）
    from transformers import AutoModelForCausalLM, AutoTokenizer
    tokenizer = AutoTokenizer.from_pretrained("Qwen/Qwen-1.8B-Chat", trust_remote_code=True)
    model = AutoModelForCausalLM.from_pretrained("Qwen/Qwen-1.8B-Chat", trust_remote_code=True).to("cpu")
    response, _ = model.chat(tokenizer, prompt, history=[])
    return response
```

（4）核心接口（用户交互）

Python

```python
from fastapi import FastAPI, Body
from fastapi.middleware.cors import CORSMiddleware

app = FastAPI(title="智保灵犀增强RAG服务")
# 跨域配置（适配前端调用）
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# 多轮对话接口
@app.post("/api/chat")
async def chat(
    user_id: str = Body(..., description="用户唯一标识"),
    query: str = Body(..., description="用户提问")
):
    # 1. 获取历史上下文
    context = get_conversation_context(user_id)
    # 2. 调用RAGflow检索知识库
    ragflow_result = call_ragflow(query)
    if "error" in ragflow_result:
        return {"code": 500, "message": ragflow_result["error"], "data": None}
    # 3. 生成答案
    answer = generate_answer(query, ragflow_result, context)
    # 4. 保存新上下文
    save_conversation_context(user_id, query, answer)
    # 5. 记录交互日志
    save_interaction_log(user_id, query, answer)
    # 6. 返回结果
    return {
        "code": 200,
        "message": "success",
        "data": {
            "answer": answer,
```

```
            "context_count": len(context) + 1,  # 当前对话轮数
            "source": [meta.get("source") for meta in ragflow_result.get("metadatas", [])]  # 知识来源
        }
    }
```

（三）存储层设计（轻量化）

存储类型	组件	存储内容	配置优化（适配 2 核 8G）
上下文缓存	Redis 7.0	多轮对话上下文（用户 ID- 对话历史）	最大内存限制 2G，采用 LI 策略，过期时间 30 分钟
交互 / 合规日志	SQLite 3	用户提问、答案、合规检测结果	单文件数据库，每日自动备 志保留 30 天
配置文件	本地 JSON 文件	RAGflow API 密钥、违规词库等	加密存储（AES-128），仅 部读取

四、部署方案（适配阿里云服务器）

1. 服务器配置要求

配置项	最低要求	推荐配置
CPU	2 核（vCPU）	4 核（vCPU）
内存	8 GiB	16 GiB
硬盘	ESSD 40G	ESSD 80G（预留 备份空间）
网络	公网 IP，开放 9000（RAGflow）、8000（增强 RAG）端口	-

2. 部署步骤

步骤 1：部署 RAGflow（独立容器）

```bash
# 1. 安装Docker Compose
curl -L "https://github.com/docker/compose/releases/download/v2.24.6/docker-compose-$(uname -s)-$(uname -m)" -o /usr/local/bin/docker-compose
chmod +x /usr/local/bin/docker-compose

# 2. 克隆RAGflow并启动
git clone https://github.com/infiniflow/ragflow.git
cd ragflow
# 修改docker-compose.yml，调整资源限制（如上文）
docker-compose up -d

# 3. 初始化RAGflow：访问http://IP:9000，创建重疾险知识库，上传文档，生成API密钥
```

步骤 2：部署增强 RAG 服务

```bash
# 1. 创建项目目录
mkdir -p /data/rag-enhanced
cd /data/rag-enhanced

# 2. 编写依赖文件requirements.txt
cat > requirements.txt << EOF
fastapi==0.104.1
uvicorn==0.24.0
redis==5.0.1
requests==2.31.0
pydantic==2.4.2
transformers==4.35.2
torch==2.1.0
EOF

# 3. 安装依赖
pip3 install -r requirements.txt

# 4. 上传增强RAG服务代码（main.py、utils.py等）
# 5. 启动Redis（容器化）
docker run -d --name redis -p 6379:6379 --memory=2g redis:7.0 --maxmemory 2gb --maxmemory-policy allkeys-lru

# 6. 启动增强RAG服务
uvicorn main:app --host 0.0.0.0 --port 8000 --workers 1 --limit-concurrency 10
```

步骤 3：前端部署（可选）

- 前端采用 Vue3 + Element Plus 开发极简交互界面，打包后通过 Nginx 部署（占用端口 80）；
- 核心功能：用户 ID 输入、提问框、答案展示、对话历史查看。

五、核心性能与合规保障

1. 性能优化（适配 2 核 8G）

- **RAGflow 优化**：关闭非核心功能（如 OCR、多模型对比），仅保留文档解析 + 向量化 + 检索；
- **增强 RAG 优化**：LLM 推理使用 CPU 模式（无 GPU），限制并发数≤10，避免资源耗尽；
- **缓存优化**：高频问题（如“重疾险等待期”）答案缓存至 Redis，有效期 1 小时，减少 RAGflow 调用次数。

2. 合规保障

- **违规词库实时更新**：支持通过配置文件动态添加保险违规词汇，无需重启服务；
- **溯源能力**：所有答案均关联 RAGflow 知识库来源，可点击查看原文片段；
- **日志审计**：记录所有用户交互与合规检测结果，支持导出审计报告。

六、测试与演示流程

1. 功能测试

测试场景	测试用例	预期结果
单轮检索	提问“重疾险甲状腺结节核保规则”	返回匹配的知识容，无违规表述源
多轮对话	第一轮“重疾险等待期多久”→第二轮“等待期内出险怎么办”	上下文连贯，第案基于第一轮展
违规表述检测	提问“买这款重疾险能保证赔付吗”	答案中“保证赔屏蔽，记录合规
RAGflow 异常处理	停止 RAGflow 服务后提问	返回友好错误提服务崩溃

2. 演示流程

1. 访问 RAGflow 后台（http://IP:9000），展示知识库文档上传、分块、向量化过程；
2. 访问增强 RAG 前端界面（http://IP:80），输入用户 ID，发起多轮保险相关提问；
3. 展示答案溯源、合规声明、对话历史保留功能；
4. 查看交互日志与合规日志，演示审计能力。

七、总结

核心优势

1. **解耦架构**：RAGflow 专注知识库管理，增强 RAG 专注业务交互，各自独立部署 / 迭代；
2. **轻量化**：适配中小服务器资源，无复杂依赖，部署成本低；
3. **场景适配**：深度适配保险领域，支持合规校验、多轮对话，满足智保灵犀核心需求。

后续迭代方向

1. 集成 OCR 能力，支持扫描件保险文档解析；
2. 优化 LLM 推理性能，接入 GPU 加速；
3. 增加用户权限分级，支持多角色访问控制。